Medical Image Processing

Denis Molnar Ardelean

May 26, 2026

https://github.com/Randenlomnis0/cuddly-octo-guacamole

Contents

1 DICOM Loading and Visualization

Our first step is to load and visualize the images. To achieve this, we first tried using the program *3D Slicer*, but we weren't able to make it work with the dynamic PET scan. We suspect that separating each frame or modifying the metadata would've helped fix this.

We eventually decided to use the Python library *Pydicom*, equipped for handling DICOM files. Here we can see how the images are loaded and preprocessed.

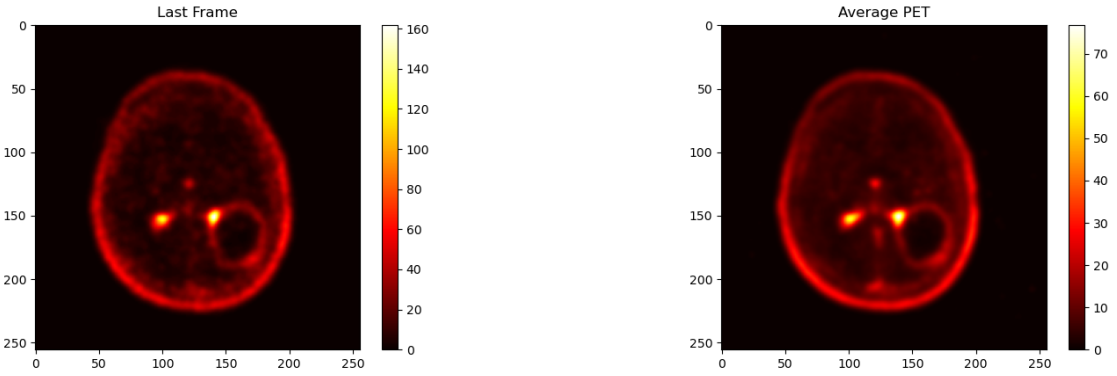
```
ds_pet = pydicom.dcmread("./FORISI/02324177
   _s2_e_1_BRAIN_DINAMIC_COLINA_AC_FORISI260916")
ds_mr = pydicom.dcmread("./FORISI/15252129
   _s1_AX_3D_T1__C_FSPGR_FORISI260916")

pet_frames = int(ds_pet.get((0x0028, 0x0008), None).value)
pet_slices = ds_pet.get((0x0054, 0x0081), None).value
pet_rows = ds_pet.Rows
pet_cols = ds_pet.Columns
pet_slice_thickness = ds_pet.SliceThickness
pet_pixel_spacing = ds_pet.PixelSpacing

mr_frames = int(ds_mr.get((0x0028, 0x0008), None).value)
mr_slices = ds_mr.get((0x0054, 0x0081), None).value
mr_rows = ds_mr.Rows
mr_cols = ds_mr.Columns
mr_slice_thickness = ds_mr.SliceThickness
mr_pixel_spacing = ds_mr.PixelSpacing

ds_pet = ds_pet.pixel_array.reshape(pet_frames // pet_slices,
   pet_slices, pet_rows, pet_cols)[-1, ::-1, :, :]
ds_mr = ds_mr.pixel_array.reshape(mr_frames // mr_slices, mr_slices,
    mr_rows, mr_cols)[-1, ::-1, :, :]

ds_pet = resample(ds_pet, (pet_slice_thickness, float(
   pet_pixel_spacing[0]), float(pet_pixel_spacing[1])))
ds_mr = resample(ds_mr, (mr_slice_thickness, float(mr_pixel_spacing
   [0]), float(mr_pixel_spacing[1])))

ds_pet = resize_to_cube(ds_pet)
ds_mr = resize_to_cube(ds_mr)

ds_pet = (normalize(ds_pet) / 255).astype(np.float32)
ds_mr = (normalize(ds_mr) / 255).astype(np.float32)
```

The preprocessing is comprised of the following steps:

- Reorganizing the voxels so that they are shaped correctly.
- Resampling the voxels so that we have them in isotropic voxels instead of anisotropic ones (this step could be skipped but is convenient for visualization purposes).

- We convert every frame into a cube, meaning every dimension has the same size. This is done by adding padding and resizing.
- The last step is normalization of voxel intensities. This improves visualization and might help later in coregistration.

In Figure 1, we can see the axial median plane of the last frame of the dynamic PET (left), and the average of all frames (right).



(a) Last frame

(b) Average of all frames

Figure 1: Axial median plane comparison of dynamic PET images.

In the repository, the code for visualizing the 3 median planes across time is pretty self-explanatory, and the results can't really be shown in a PDF, so we don't explain in detail here. The result is in the file called *pet_dynamic_median.gif*.

2 3D Rigid Coregistration

For co-registration we used the Python library *SimpleITK*, an open-source multi-dimensional image analysis library developed mainly for biomedical sciences.

We preprocess our voxel arrays in the same way we described in the previous section.

After that, we convert our voxel arrays into ITK images. ITK images are sets of points that occupy a physical region in space.

```
fixed = sitk.GetImageFromArray(ds_mr.astype(np.float32))
moving = sitk.GetImageFromArray(ds_pet.astype(np.float32))
```

The next step is to initialize a starting transformation. We choose to align the centers of the voxel grids for our initial guess, and the transformation we look for is comprised of a rotation and a translation.

```
initial_transform = sitk.CenteredTransformInitializer(
    fixed,
    moving,
```

```
    sitk.Euler3DTransform(),
    sitk.CenteredTransformInitializerFilter.GEOMETRY
)
```

We must create an *ImageRegistrationMethod* object, which is responsible of the whole co-registration process.

```
reg = sitk.ImageRegistrationMethod()
```

It is at this point that we make two important choices. We define the similarity metric as Mattes Mutual Information and we set the Optimizer as a regular-step gradient descent. Mutual Information algorithm are great for multi-modal image registration, which is what we are doing.

```
reg.SetMetricAsMattesMutualInformation(numberOfHistogramBins=50)

reg.SetOptimizerAsRegularStepGradientDescent(
    learningRate=1.0,
    minStep=1e-4,
    numberOfIterations=1000,
    relaxationFactor=0.5
)
```

Before we actually do the co-registration, we define an interpolator for the *ImageRegistrationMethod* object to use internally. This is used for approximating values in between voxels.

```
reg.SetInterpolator(sitk.sitkLinear)
```

Now we perform the co-registration, which takes almost 10 minutes on our personal laptop.

As for the results, we have a nice GIF containing a rotating Maximum Intensity Projection on the coronal-sagittal plane. Since this can't be viewed in a PDF, we suggest taking a look at the file *coregistration.gif* from the repository. We leave here a screenshot, figure 2, of a frame of said GIF.

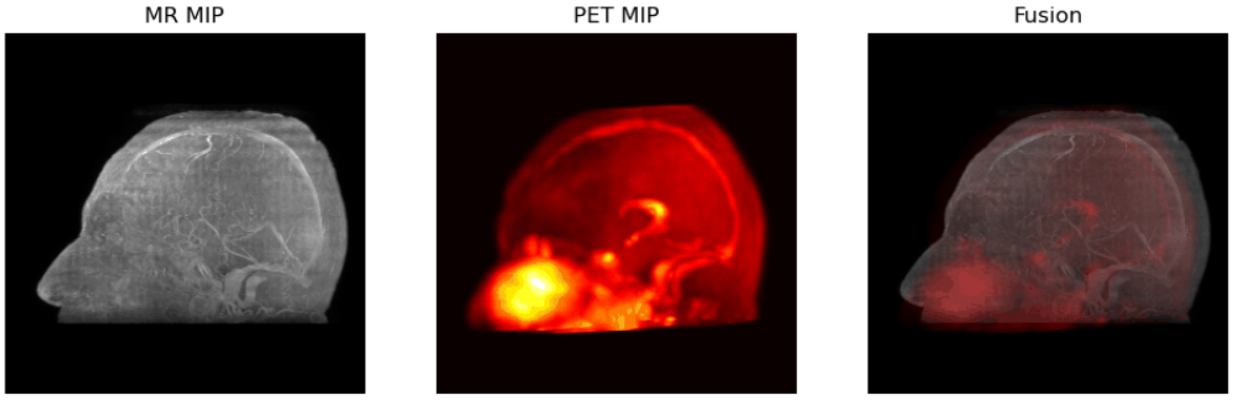


Figure 2: Screenshot of the rotating MIP of the coronal-sagittal plane.

3 3D Image Segmentation

The first step is to find the bounding box for the tumor that we wish to segment. We accomplish this thanks to the plots from *matplotlib* conveniently showing the pixel that the mouse is hovering over. This will be used as input for the segmentation model. In figure 3 we see our chosen bounding box for the PET.

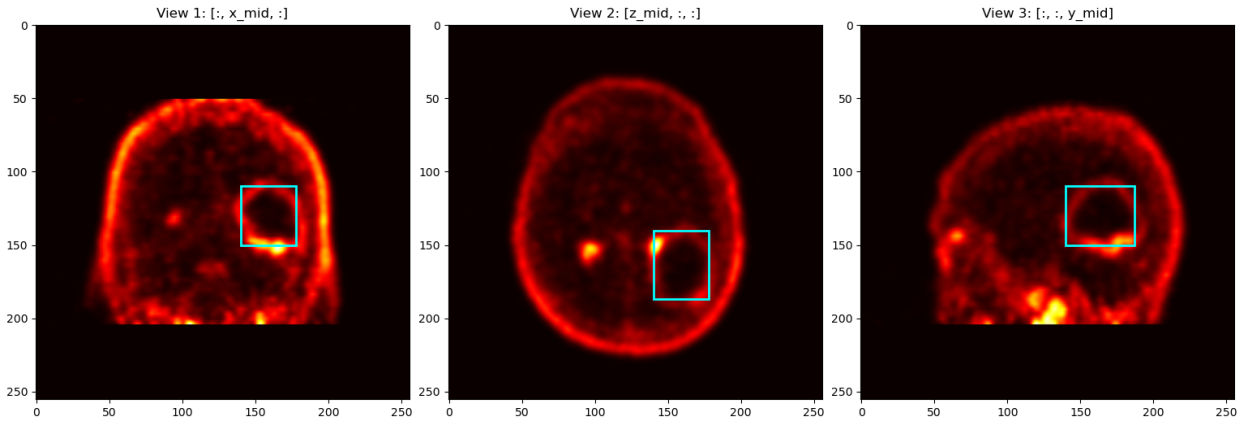


Figure 3: Bounding box of the tumor in the PET scan (last frame).

Ideally, we would use the bounding box we got for the PET scan and the transformation we obtained during the co-registration to automatically obtain the bounding box for the MR image. However, we though it would be less error-prone to just select the bounding box directly in the MR image. Both options involve the same manual labor, and if we had a reason to use the first option, then implementing it wouldn't be too hard anyways. Worst case scenario, by obtaining the bounding box manually we have a convenient tool for verifying the correctness of our results. In figure 4 we see our chosen bounding box for the MR.

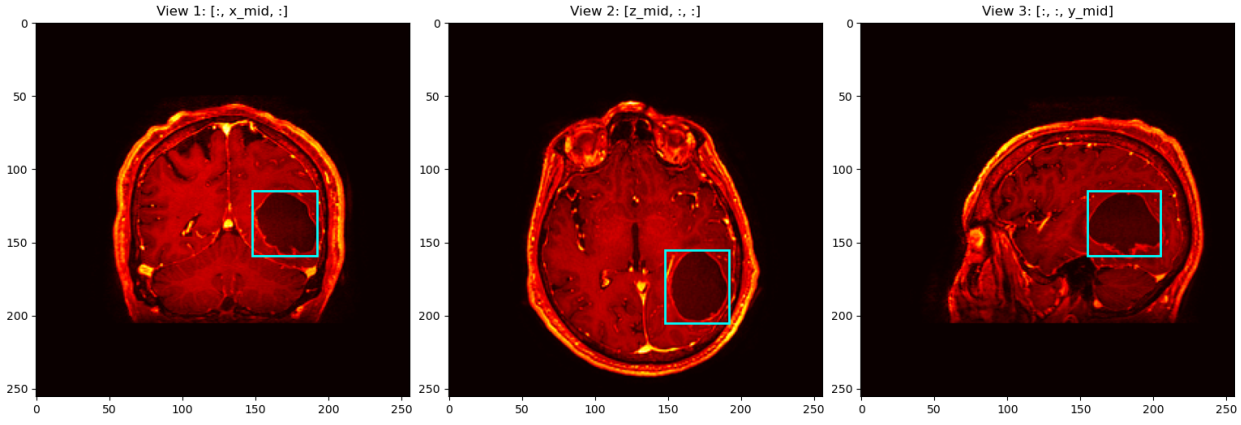


Figure 4: Bounding box of the tumor in the MR scan.

For the actual segmentation we use the Segment Anything Model (SAM) with a Vision Transformer backbone (ViT-H), a general-purpose image segmentation model developed to identify and outline objects in images without being trained for a specific task beforehand. It was created by Meta AI in 2023.

We use this model with a set of pretrained weights, which is necessary for running this in reasonable amounts of time.

In the code we only segment 5 slices because of execution time reasons, but it could be changed very easily for a real-life scenario.

This time we can actually show the results in figure 5, as they are an image, rather than a GIF.

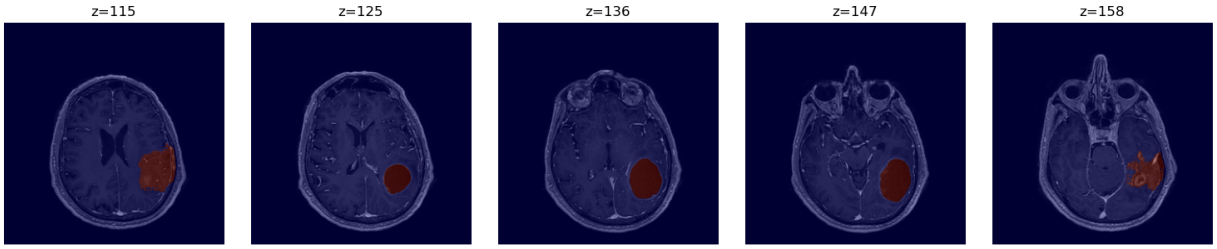


Figure 5: Axial median plane comparison of dynamic PET images.

The results are clearly good, except for the first few and last few considered slices, as the tumor isn't visible on those. We didn't implement a numerical measure of the segmentation's performance, as we don't really have anything objectively correct to compare it to.